

■ ALBEREI · DOCUMENT 03

Use Cases.

*Five product shapes worked through end to end.
Pick the one closest to yours and copy the pattern.*

AUDIENCE

VERSION

ISSUED

Product, engineering

1.0

May 2026

A humour module for the conversational loop.

Robots that talk earn their place in a home by knowing when to be quiet, when to be useful, and occasionally when to be funny. The third is the hardest, and the one most often left out. Alberei is the module a robot calls when the conversational context invites colour: a routine apology, an idle minute, an answer to a small-talk cue.

Configuration

REGISTER

`warm` for companion robots; `deadpan` for task robots

LENGTH

`short` (spoken delivery)

COUNT

2 (server picks, discards the other)

WHEN TO CALL

Small-talk classifier, silence over 1500 ms, recurring apology event

WHEN NOT TO CALL

Safety-critical instructions, emergency, explicit user complaint

Pseudo-pattern

```
on_idle_or_smalltalk(ctx):
    if ctx.user_signal == 'small_talk' or ctx.silence_ms > 1500:
        items = alberei.generate(
            lang=ctx.lang, context=ctx.summary(),
            register='warm', length='short', count=2)
        speak(items[0].text)
        log(ctx.session_id, items[0].id, items[0].structure)
```

Inaugural pilot. Phi-Yo Embodied, Llull Lab's wellbeing companion robot, ships with Alberei as its humour module. Register `warm`, length `short`, per-session humour cap to keep the effect scarce and intentional.

Wit inside the synthesis budget.

Voice assistants have a hard latency budget: from end of user speech to start of agent speech, under one second feels alive, under two feels acceptable, over two feels broken. Alberei generation must finish inside the assistant's TTS preparation window.

Configuration

REGISTER

Persona-locked at product setup. `warm` or `deadpan` read most naturally over speech.

LENGTH

`short` always

COUNT

1 to 2; surfaces are auditory and brief

TIMEOUT

800 ms; fall through to non-humorous reply on miss

AUDIENCE

Per-household tag, no PII; useful for retrospective register tuning

Edge

The classifier for "this turn invites humour" is the engineering challenge, not the API call. Over-trigger and the assistant becomes a stand-up act. Under-trigger and the integration is invisible. Start strict, loosen with usage data.

Humour as a softener, used sparingly.

A humour line in a refund-pending flow derails trust. A humour line in a "your delivery is delayed by twenty minutes" message softens a wait. The decision is not whether the bot can be funny; it is whether this specific turn is one where humour helps.

Configuration

REGISTER

in all cases. Customer service is not the place for .

LENGTH

COUNT

1; the bot speaks, the user does not pick

GATING

Per-turn classifier with explicit allow-list of flow types

REFUSAL HANDLING

On 422, send the non-humorous version of the message

Suggested allow-list

- Delivery delay (under one hour)
- Service-status all-clear after an incident
- Greeting in the first message of a session
- Goodbye in a successful session

Suggested deny-list

- Any refund, payment, or billing flow
- Any complaint or escalation
- Any session involving an account-security topic
- Any session flagged as a vulnerable user

Pre-generate at content time, sample at runtime.

Game design budgets are about quota efficiency. The hot path is the player turn; the API call cannot live there. Pre-generation at content-build time lets the runtime sample without calling Alberei at all.

Configuration

REGISTER

Per-NPC. Use the NPC's voice document to lock the choice.

LENGTH

Per-beat, usually

COUNT

5 (the maximum); cache all five

CACHE KEY

REFRESH

On content update, on player feedback metrics drop

Build-time pseudo-pattern

```
for npc in cast:
    for beat in npc.beats:
        for lang in supported_languages:
            items = alberei.generate(
                lang=lang, context=beat.context,
                register=npc.register, length='short', count=5)
            cache.put((lang, npc.id, npc.register, beat.id), items)
```

Runtime sampling is free. The quota cost is bounded by the size of the content surface, not by the number of players or hours played.

Suggestions the author chooses from.

Writing tools have an author in the loop. The author wants options, sees them, picks one. The integration is the most generous of the five: show all items, surface the structure tag, let the author ask for a different mechanism.

Configuration

REGISTER

User-selectable; default to the document's prevailing register

LENGTH

User-selectable; default

COUNT

3 to 5

UI

Sidebar list; each item shows the structure tag and a "regenerate with different mechanism" action

TELEMETRY

Track which structures the author picks; surface back as register-tuning hints

Suggested UI copy

- **Add humour:** primary action on the selected paragraph
- **Tone:** dropdown with the four registers, plain English labels
- **Why this works:** hover reveals the structure tag in plain language
- **Regenerate:** requests a different structure for the same context

Common mistakes, across product shapes.

- **Calling on every turn.** The user habituates within minutes.
- **Mixing registers in a session.** The user cannot name what is wrong, but it is wrong.
- **Caching per user-facing utterance.** The second time a line lands, it loses.
- **Showing a refusal as a moderation banner.** The product looks anxious. Skip silently.
- **Translating output between languages.** Generate native, every time, for every language you support.
- **Tying register switches to user mood signals.** Persona instability erodes trust faster than the right register provides delight.